

BLOC 4 - DOSSIER DE VALIDATION

Anonym

Maintenir l'application logicielle en condition opérationnelle

Dossier écrit compact, compatible avec un dépôt Canvas : supervision, traitement des anomalies, maintenance, journal de version et collaboration support.

Candidat	Lukas Bouhlel
Version	v1.0.0-bloc4
Date	7 juillet 2026
Périmètre	Backend Express, frontend React, application Flutter, MySQL, Docker, VPS, CI/CD et supervision.

Dossier de validation - Bloc 4

Sommaire

- Présentation du périmètre maintenu
- Processus de mise à jour des dépendances
- Système de supervision et d'alerte
- Collecte et consignation des anomalies
- Fiche de consignation d'une anomalie
- Traitement d'une anomalie détectée
- Recommandations argumentées d'amélioration
- Journal de version
- Problème résolu avec le support client
- Synthèse de validation du bloc 4

1. Présentation du périmètre maintenu

Anonym est une application sociale web et mobile orientée messagerie temps réel, confidentialité, boutique virtuelle, inventaire, facturation et administration. Le périmètre maintenu couvre le backend Node.js / Express, l'API REST, Socket.IO, la base MySQL gérée par Sequelize, le frontend web React / Vite, l'application mobile Flutter, l'infrastructure Docker Compose sur VPS, Nginx, Certbot, les workflows GitHub Actions et les outils de supervision.

L'objectif de maintenance est de garantir la disponibilité de l'application, la sécurité des dépendances, la traçabilité des anomalies et la capacité à déployer rapidement un correctif contrôlé.

Élément	Technologie	Rôle en maintenance
Backend	Node.js, Express, Socket.IO	API, temps réel, sécurité, logs, healthchecks
Données	MySQL, Sequelize, migrations	Persistance, évolutions de schéma, sauvegarde
Web	React, Vite, Axios, React Query	Interface publique, support, administration
Mobile	Flutter, Dio, Provider, go_router	Application utilisateur, support, feedback, notifications
Infra	Docker Compose, Nginx, Certbot, VPS	Déploiement, HTTPS, reverse proxy, persistance
CI/CD	GitHub Actions	Tests, audit, build, déploiement préproduction et production
Supervision	Healthchecks, Prometheus, pino, Artillery, OVH/Site24x7	Disponibilité, performance, alertes et diagnostic

Les preuves projet utilisées dans ce dossier sont notamment `app/utils/observability.js`, `app/tests/test_unitaires/observability.test.js`, `app/tests/performance/health.yml`, `.github/workflows/ci.yml`, `.github/workflows/ci-cd.yml`, `.github/workflows/cd.yml`, `docker-compose.prod.yml`, `docs/exploitation-stockage-vps.md` et les commits de correction récents.

2. Processus de mise à jour des dépendances

2.1 Périmètre logiciel concerné

Le processus concerne toutes les dépendances qui peuvent introduire une faille, une régression ou une incompatibilité d'exécution.

Périmètre	Fichiers suivis	Outils de contrôle	Fréquence
Backend Node.js	<code>anonym-back-end/package.json</code> , <code>package-lock.json</code>	Snyk, <code>npm audit</code> , Jest, ESLint	À chaque push/PR, revue mensuelle
Frontend React	<code>anonym-front-end/package.json</code> , <code>package-lock.json</code>	Snyk, <code>npm audit</code> , Jest, ESLint, Stylelint, build Vite	À chaque push/PR, revue mensuelle
Mobile Flutter	<code>anonym_front_flutter/pubspec.yaml</code> , <code>pubspec.lock</code>	SBOM CycloneDX, Snyk SBOM, <code>flutter analyze</code> , <code>flutter test</code>	À chaque CI, revue mensuelle
Docker et CI	Dockerfiles, workflows GitHub Actions, images Docker	GitHub Actions, revue manuelle, déploiement préprod	Revue mensuelle et avant livraison
Services externes	Stripe, Firebase, Mapbox, SMTP	Tests fonctionnels, secrets GitHub, configuration par environnement	Revue trimestrielle ou changement fournisseur

2.2 Type de mise à jour

La surveillance est automatisée, mais l'intégration reste manuelle et sécurisée. Les audits Snyk, `npm audit`, SBOM Flutter et tests CI signalent les risques automatiquement. La montée de version est ensuite faite dans une branche dédiée, relue, testée puis validée en préproduction avant production. Aucune mise à jour majeure n'est fusionnée automatiquement, car elle peut impacter l'authentification, les cookies, Socket.IO, les paiements ou le build mobile.

Type	Exemple	Décision
Patch sécurité	Correctif npm, package Dart, action GitHub vulnérable	Traitement prioritaire sous 24 à 48 h si criticité élevée
Mineure compatible	Bibliothèque sans rupture documentée	Intégration groupée lors de la revue mensuelle
Majeure	Express, Vite, Flutter SDK, Socket.IO, Sequelize	Analyse d'impact, branche dédiée, tests élargis, validation préprod
Image Docker	mysql , nginx , certbot	Version à pinner puis mise à jour planifiée

2.3 Chaîne de validation

- Détection par CI, Snyk, npm audit , SBOM Flutter ou veille technique.
- Qualification de l'impact : sécurité, compatibilité, performance, coût de test.
- Création d'une branche fix/dependency-* ou chore/dependency-* .
- Mise à jour des fichiers de verrouillage : package-lock.json ou pubspec.lock .
- Exécution locale ciblée : lint, tests unitaires, tests fonctionnels, build.
- Passage en CI : sécurité, Flutter, backend/frontend, performance Artillery.
- Déploiement en préproduction si le changement touche l'exécution.
- Déploiement production via main , puis contrôle /health , /metrics , logs et support.
- Ajout au journal de version avec correctifs, preuves et validation.

Ce processus répond à C4.1.1 car il précise la fréquence, le périmètre et le type de mise à jour. Il limite le risque d'obsolescence tout en évitant les mises à jour automatiques non maîtrisées.

3. Système de supervision et d'alerte

3.1 Périmètre de supervision

La supervision couvre les composants qui conditionnent l'accès utilisateur : API backend, endpoints publics, Socket.IO, MySQL, Nginx/HTTPS, stockage des uploads, CI/CD, espace disque VPS, certificats TLS et support utilisateur. Le système est adapté à une application web/mobile avec backend centralisé, car il surveille l'API, les métriques HTTP, les erreurs applicatives et la disponibilité serveur.

3.2 Sondes mises en place

Sonde	Source	Indicateurs	Seuils / réaction	Finalité
Santé backend	/health , /status , /api/health	status , service, version, uptime, mémoire, runtime	HTTP différent de 200 ou status != ok : incident disponibilité	Confirmer que l'API répond
Métriques Prometheus	/metrics avec prom-client	Requêtes HTTP, durée, codes statut, métriques Node.js	P95 > 750 ms, max > 2 s, erreurs > 1 % : analyse perf	Suivre qualité et performance
Logs structurés	pino-http	requestId , route normalisée, méthode, statut, niveau warn/error	5xx ou pics 4xx : diagnostic et création d'anomalie	Identifier la cause technique
Base de données	Healthcheck Docker mysqladmin ping	État conteneur MySQL	5 échecs de ping : conteneur non sain	Vérifier la persistance
Test performance CI	Artillery health.yml	Erreur max, P95, temps max sur health/metrics	maxErrorRate <= 1 , P95 <= 750 ms, max <= 2000 ms	Empêcher une livraison dégradée
CI/CD	GitHub Actions	Jobs sécurité, tests, build, déploiement	Échec job : blocage merge/déploiement	Éviter les régressions
Stockage VPS	df -h , script cleanup	% disque, backups, cache, Docker	Alerte à 85 %, critique à 90 %	Prévenir les pannes de déploiement
Supervision externe	OVH / Site24x7	Disponibilité serveur, uptime HTTP	Indisponibilité détectée : notification support/tech	Surveillance hors application
Certificats	Certbot renouvellement 12 h	Échéance TLS, renouvellement	Échec renouvellement : alerte exploitation	Maintenir HTTPS

3.3 Critères de qualité et performance

Les critères retenus sont liés aux usages réels : l'utilisateur doit pouvoir ouvrir l'application, se connecter, échanger des messages, charger des médias et finaliser un paiement. Le backend doit répondre rapidement, conserver ses fichiers, exposer ses métriques et rester observable en cas d'erreur.

Critère	Objectif projet	Contrôle
Disponibilité API	API joignable en continu	/health , supervision externe, Docker ps
Performance API	P95 inférieur à 750 ms sur endpoints techniques	Artillery et histogramme Prometheus
Fiabilité des médias	Uploads persistants après redéploiement	Volume Docker ./anonym-back-end/uploads:/app/uploads
Sécurité	Pas de vulnérabilité haute connue en livraison	Snyk, npm audit , SBOM Flutter
Traçabilité	Erreurs reliées à une route et un requestId	pino-http , logs sans secrets
Déploiement	Déploiement stoppé si dépôt serveur modifié	Workflows ci-cd.yml et cd.yml

3.4 Modalité de signalement

Les alertes techniques arrivent par GitHub Actions, par la supervision externe ou par les logs serveur. Les anomalies utilisateur arrivent par la page Support web, l'écran Support/Feedback mobile et la route /api/admin/report . Chaque signalement est trié selon quatre niveaux : S1

indisponibilité totale, S2 fonction majeure dégradée, S3 bug contournable, S4 amélioration. Les S1/S2 sont traitées immédiatement, les S3 sont planifiées dans le sprint ou la maintenance mensuelle.

Ce dispositif répond à C4.1.2 car il définit le périmètre de supervision, les indicateurs, les sondes et les modalités de signalement nécessaires à la disponibilité du logiciel.

4. Collecte et consignation des anomalies

4.1 Sources de collecte

Source	Exemple	Information récupérée
Support web	Formulaire <code>/support</code> , <code>POST /api/admin/report</code>	Email, type, description, utilisateur connu
Support mobile	<code>SupportScreen</code> , <code>FeedbackScreen</code> , <code>AdminRepository.report</code>	Email, type de rapport, contenu
Monitoring	<code>Site24x7/OVH</code> , <code>/health</code> , <code>/metrics</code>	Date, statut, disponibilité, latence
Logs backend	<code>pino-http</code>	Route, méthode, statut, <code>requestId</code> , niveau
CI/CD	GitHub Actions	Job échoué, commit, branche, logs de test
Tests automatisés	Jest, Flutter, Artillery	Scénario en échec, seuil dépassé
Retours internes	Recette préprod, test manuel	Étapes de reproduction et capture

4.2 Fiche de consignation standard

Chaque anomalie est consignée avec les champs suivants afin de permettre la reproduction et le choix du correctif.

Champ	Contenu attendu
Identifiant	Code unique, date, composant concerné
Environnement	Local, CI, préproduction ou production
Version	Commit, branche, tag ou version déployée
Source	Support, monitoring, logs, CI, test manuel
Gravité / priorité	S1 à S4, impact utilisateur, urgence
Description	Résumé compréhensible du problème
Étapes de reproduction	Parcours précis, données de test, compte ou rôle
Résultat attendu	Comportement normal
Résultat constaté	Comportement observé
Preuves	Logs, capture, endpoint, job CI, commit
Analyse	Cause probable ou cause racine
Préconisation	Correctif proposé, test attendu, risque
Traitement	Branche, commit, PR, déploiement, validation
Statut	Nouveau, qualifié, en cours, corrigé, déployé, clôturé

4.3 Processus de traitement

- Réception du signalement et création d'une fiche.
- Qualification : environnement, gravité, composant, utilisateurs touchés.
- Reproduction sur préproduction ou local quand c'est possible.
- Recherche de cause avec logs, métriques, commits récents et configuration.
- Choix du correctif et estimation du risque.
- Implémentation dans une branche dédiée.
- Validation par tests automatisés et scénario de non-régression.
- Déploiement préproduction, puis production si la validation est positive.
- Surveillance renforcée après déploiement.
- Mise à jour du journal de version et clôture du ticket.

Ce processus répond à C4.2.1 car il structure la collecte, impose les informations permettant de reproduire le bug et relie l'analyse à un correctif.

5. Fiche de consignation d'une anomalie rencontrée

Champ	Valeur
Identifiant	ANO-2026-07-02-001
Titre	Images uploadées non persistantes après redéploiement production
Source	Retour support utilisateur et vérification technique après déploiement
Environnement	Production, backend Docker <code>anonym-back-end</code>
Composants	Backend Express, stockage <code>uploads</code> , Docker Compose, Nginx statique
Gravité	S2 : fonction majeure dégradée, médias utilisateur indisponibles
Version concernée	Production avant commit <code>62a03902</code>
Description	Les images de profil, images de messages ou contenus stockés dans <code>uploads</code> peuvent devenir indisponibles après reconstruction du conteneur backend.
Étapes de reproduction	Uploader une image, vérifier son affichage, lancer un redéploiement Docker production, revenir sur le profil/message, constater que le fichier n'est plus servi.
Résultat attendu	Les fichiers uploadés restent disponibles après redéploiement.
Résultat constaté	Le chemin applicatif existe en base, mais le fichier physique peut ne plus exister dans le nouveau conteneur.
Analyse	Le dossier <code>/app/uploads</code> était porté par le filesystem éphémère du conteneur backend au lieu d'être monté depuis l'hôte.
Préconisation	Ajouter un volume Docker de production pour persister <code>uploads</code> sur le VPS et vérifier le comportement après redéploiement.
Preuve projet	Commit <code>62a03902</code> : Persist production uploads volume , fichier <code>docker-compose.prod.yml</code> .
Statut	Corrigée et documentée dans le journal de version.

Correctif appliqué dans `docker-compose.prod.yml` :

```
backend:
  volumes:
    - ./anonym-back-end/uploads:/app/uploads
```

6. Traitement d'une anomalie détectée

Le traitement de l'anomalie ANO-2026-07-02-001 a suivi le processus d'intégration et de déploiement continu du projet.

Étape	Action réalisée	Preuve / résultat
Qualification	Impact sur la disponibilité des médias utilisateurs après redéploiement	Anomalie classée S2
Analyse cause racine	Vérification de <code>docker-compose.prod.yml</code> : absence de volume <code>uploads</code> pour le backend production	Cause Docker identifiée
Correction	Ajout du bind mount <code>./anonym-back-end/uploads:/app/uploads</code>	Commit <code>62a03902</code>
Branche / revue	Branche <code>fix/prod-uploads-volume</code> , PR de livraison vers préproduction puis production	PR #172 / #173 visibles dans l'historique
Validation technique	Redéploier, vérifier <code>/health</code> , tester upload, contrôler que le fichier reste disponible après rebuild	Non-régression média
Déploiement	Pipeline GitHub Actions, Docker Compose production, vérification conteneurs	<code>cd.yml</code> , <code>docker-compose.prod.yml</code>
Surveillance	Contrôle logs backend, endpoints <code>/health</code> et <code>/metrics</code> , retours support	Incident clôturé

La correction tire profit de la CI/CD car elle passe par les branches de livraison, les jobs de contrôle, le déploiement VPS et la surveillance post-déploiement. Elle résout l'anomalie en séparant les données utilisateurs du cycle de vie du conteneur applicatif.

Mesure de prévention ajoutée au processus : tout nouveau stockage applicatif doit être classé comme éphémère ou persistant avant déploiement. Les fichiers utilisateurs, dumps, factures ou médias doivent être montés en volume, sauvegardés ou externalisés.

7. Recommandations argumentées d'amélioration

Recommandation	Argument	Gain attendu	Coût / délai	Priorité
Mettre en place Prometheus + Grafana + Alertmanager ou équivalent managé	Les métriques existent déjà via <code>/metrics</code> , il manque une visualisation continue et des alertes formalisées	MTTR réduit, historique P95/5xx/mémoire	Moyen, 2 à 4 jours	Haute
Externaliser les uploads vers un stockage objet compatible S3/OVH	Le volume local corrige la persistance, mais reste lié au VPS	Meilleure durabilité, backups simplifiés, scalabilité	Moyen, 3 à 5 jours	Haute
Ajouter Dependabot ou Renovate avec PR groupées	La surveillance existe, mais l'ouverture des PR peut être automatisée	Dette sécurité réduite, suivi régulier	Faible, 1 jour	Haute
Pinner les images Docker (<code>mysql:8.0</code> , <code>nginx</code> , <code>certbot</code>)	<code>latest</code> peut changer sans maîtrise et introduire une régression	Déploiements reproductibles	Faible, 0,5 à 1 jour	Haute
Enrichir le formulaire support	Ajouter version app, OS, navigateur, endpoint, capture optionnelle, <code>requestId</code>	Reproduction plus rapide des bugs	Faible à moyen, 1 à 2 jours	Moyenne
Créer un runbook incident	Les actions existent, mais doivent être centralisées pour S1/S2	Diagnostic plus rapide, passation plus simple	Faible, 1 jour	Moyenne
Automatiser le changelog depuis commits/PR	Le journal est manuel	Traçabilité plus fiable et moins coûteuse	Faible, 1 jour	Moyenne
Ajouter une sauvegarde/restauration testée MySQL + uploads	Le projet documente le stockage VPS, mais la restauration doit être répétée	Réduction du risque de perte de données	Moyen, 2 à 3 jours	Haute

Ces recommandations sont réalistes car elles s'appuient sur des briques déjà présentes : métriques Prometheus, CI/CD, Docker, support, logs et documentation VPS. Elles renforcent l'attractivité du logiciel en améliorant la fiabilité perçue, la rapidité de correction et la confiance utilisateur.

8. Journal de version

Le journal ci-dessous est un journal d'exploitation interne construit à partir des commits et PR du projet. Le package backend reste en `1.0.0`, mais les versions ci-dessous documentent les livraisons de maintenance.

Version	Date	Référence	Améliorations / correctifs	Validation
v1.0.4	2026-07-03	<code>cde0e633</code> , PR #177, <code>a96591e5</code> , <code>9a999ac6</code>	Couverture globale de tests, correction placeholder message mobile, lien CGU inscription, création groupe avec image	Tests Flutter/Jest et CI
v1.0.3	2026-07-02	<code>62a03902</code> , PR #172/#173	Persistance des uploads production via volume Docker	Redéploiement + vérification médias
v1.0.2	2026-07-02	<code>a92fcf7b</code>	Correctifs paiement mobile, URL médias relatives, session après inscription, images distantes	Tests auth et vérification mobile
v1.0.1	2026-07-01	<code>2176bfdd</code>	Correction de vulnérabilités dépendances	Snyk et <code>npm audit</code>
v1.0.0	2026-06-28	<code>af7d8d1c</code> , <code>3f934468</code> , <code>2a838fc6</code>	Stabilisation des gates de release, installations npm déterministes, timeout Docker Compose augmenté	CI/CD préprod/prod
v0.9.9	2026-06-27	<code>44d2f908</code>	Durcissement des workflows de déploiement serveur	Déploiement contrôlé

Chaque ligne contient les améliorations, les anomalies corrigées et le mode de validation. Ce journal répond à C4.3.2 car il documente les correctifs et permet de suivre les évolutions déployées.

9. Problème résolu en collaboration avec le support client

9.1 Contexte du retour client

Un utilisateur signale via le support que certaines images ne s'affichent plus après une mise en ligne. Le support reçoit la demande via le formulaire web/mobile, qui transmet l'email, le type et la description à `/api/admin/report`. Le problème est fonctionnellement visible : le message ou le profil référence encore l'image, mais l'image n'est plus accessible.

9.2 Contribution des parties prenantes

Partie prenante	Contribution
Utilisateur	Décrit le problème, fournit le contexte et le moment d'apparition
Support	Qualifie le retour, vérifie le compte, demande une reproduction simple, transmet l'anomalie
Développeur	Analyse les logs, la configuration Docker, le stockage <code>uploads</code> et l'historique de déploiement
CI/CD	Valide les tests et sécurise la livraison via branches préprod/prod
Exploitation	Redéploie, contrôle <code>/health</code> , <code>/metrics</code> , Docker Compose et disponibilité des médias

9.3 Résolution apportée

La cause racine est liée à la persistance des fichiers dans Docker. Les fichiers uploadés étaient stockés dans le conteneur backend, alors qu'un conteneur reconstruit ne doit pas être considéré comme stockage durable. Le correctif ajoute le volume `./anonym-back-end/uploads:/app/uploads` en production, puis le déploiement est validé par un test d'upload, un rebuild et une nouvelle consultation du média.

Le support peut ensuite répondre à l'utilisateur que l'incident est corrigé, que les nouveaux médias seront conservés après déploiement et que les fichiers historiques doivent être restaurés depuis sauvegarde si certains ont été perdus avant le correctif.

Cette présentation répond à C4.3.3 : le contexte client est décrit, la résolution technique est expliquée et la contribution des parties prenantes est identifiée.

10. Synthèse de validation du bloc 4

Compétence	Réponse dans le dossier	Éléments de preuve
C4.1.1 Mise à jour des dépendances	Processus, fréquence, périmètre, type automatique/manuel	Snyk, <code>npm audit</code> , SBOM Flutter, CI
C4.1.2 Supervision et alerte	Périmètre, sondes, seuils, signalement	<code>/health</code> , <code>/metrics</code> , Artillery, pino, OVH/24x7
C4.2.1 Consignation anomalies	Processus structuré et fiche complète	Support, logs, fiche ANO-2026-07-02-001
C4.2.2 Correctif et déploiement	Traitement via branches, CI/CD, Docker Compose	Commit <code>62a03902</code> , PR #172/#173
C4.3.1 Améliorations	Recommandations argumentées avec coût, délai, gain	Tableau recommandations
C4.3.2 Journal de version	Versions, correctifs, validation	Journal d'exploitation v0.9.9 à v1.0.4
C4.3.3 Support client	Exemple support avec rôles et résolution	Formulaires Support/Feedback, <code>/api/admin/report</code>

Conclusion

Le projet Anonym dispose d'une base de maintenance opérationnelle cohérente : dépendances surveillées, CI/CD structurée, endpoints de santé, métriques Prometheus, logs structurés, tests de performance, support utilisateur et journal de version. L'anomalie des uploads illustre un cycle complet : détection, consignation, analyse, correctif, déploiement et prévention. Les recommandations proposées renforcent la disponibilité, la traçabilité et la confiance utilisateur tout en restant réalistes pour la taille du projet.